FTP vs SFTP Security Analysis

A Practical Comparison of File Transfer Security

Practical cybersecurity lab | Kali Linux | Wireshark

1. Project Overview

This project investigates the security differences between File Transfer Protocol (FTP) and SSH File Transfer Protocol (SFTP). I configured both services in a Kali Linux virtual machine, transferred the same sample file, captured the network traffic with Wireshark, and compared what could be observed in each session. I also used SHA-256 hashes to check that the transferred file matched the original.

The central finding was clear: standard FTP exposed its login details and protocol commands in readable form in the packet capture, while the SFTP session was carried over SSH and appeared as encrypted traffic. This demonstrates why encrypted file-transfer methods are important when protecting credentials and data in transit.

2. Objectives

- Configure an FTP service and complete a file transfer.
- Capture FTP traffic in Wireshark and examine the information visible in the packets.
- Configure SSH/SFTP and complete the same file transfer.
- Capture SFTP traffic and compare its visibility with FTP.
- Use SHA-256 to verify the integrity of the transferred file.
- Document the setup, observations, limitations and security recommendations.

3. Lab Environment and Tools

Item	Configuration
Operating system	Kali Linux
Virtualisation	Hyper-V virtual machine
FTP service	vsftpd
Secure transfer service	OpenSSH server / SFTP
Traffic analysis	Wireshark
Capture interface	Loopback interface (lo)
Test endpoint	127.0.0.1
Integrity tool	sha256sum

Both the FTP client and server, and later the SFTP client and SSH server, ran inside the same Kali Linux virtual machine. The traffic was generated over the loopback interface (127.0.0.1) and captured on lo. This was a controlled local lab, not a test between two separate computers.

4. Preparing the Test File

I created a small text file named sample.txt containing a short test message. The file was 67 bytes. I generated its SHA-256 hash before transferring it so that I could compare the original with the received copy after the tests.

```
sha256sum /home/kali/ftp-sftp-security-lab/sample.txt
```

5. FTP Setup and Transfer

I installed vsftpd and started the service. I created a dedicated local account named labftp for the experiment and configured FTP to require local-user authentication, allow file uploads, and restrict the account to its FTP directory. Anonymous access was disabled.

During setup, vsftpd initially refused to start a session because the chroot directory was writable by the FTP user. I corrected the directory ownership and permissions: the chroot directories were owned by root and not writable by the user, while a separate upload directory remained writable. This preserved the intended restriction while allowing the file-transfer test to proceed.

I restarted the service and confirmed that it was listening on TCP port 21. I then connected to 127.0.0.1 as labftp, changed into the upload directory and uploaded sample.txt. The FTP client reported a successful transfer.

6. FTP Traffic Analysis

I opened Wireshark on the loopback interface and used the display filter tcp.port == 21. To ensure the capture contained the test session, I started the capture before connecting and transferring the file.

The capture showed FTP control-channel commands and server responses, including USER, PASS, CWD, STOR, 230 Login successful and 226 Transfer complete. Most importantly, the FTP login exchange was readable in the capture: the username and password were not protected by encryption.

This is the main security weakness demonstrated by the FTP test. Anyone able to capture the relevant unencrypted traffic on the network could potentially read the credentials and protocol activity. FTP also does not provide encryption for the file contents by default.

7. SFTP Setup and Transfer

I installed the OpenSSH server package, enabled and started the SSH service, and confirmed that it was listening on TCP port 22. I then connected with the SFTP client using the lab account.

The SFTP session opened in the user's home directory rather than the FTP-specific local root. I checked the remote working directory and navigated to the correct upload path before transferring sample.txt. The file transfer completed successfully.

8. SFTP Traffic Analysis

I captured the SFTP session on the loopback interface using the Wireshark display filter tcp.port == 22. The capture showed SSHv2 traffic, including encrypted packets. Unlike the FTP capture, the SFTP session did not expose readable FTP-style commands or the login password in the packet payload.

SFTP operates through SSH, which provides encryption for the session. This protects authentication exchanges and file-transfer data from being read directly by someone observing the network traffic, assuming SSH is configured and used securely.

9. File Integrity Verification

After the transfers, I compared the SHA-256 hash of the original sample file with the hash of the uploaded copy on the FTP server. The two hash values matched. This confirmed that the received file was identical to the original for the purpose of this test.

A matching hash supports the conclusion that the file contents were not altered during the tested transfer. Hash comparison is an integrity check; it does not itself encrypt a transfer or prove that the transfer channel was secure.

10. Results and Comparison

Security aspect	**FTP**	**SFTP**
Underlying protocol	FTP	SSH
Default port used	TCP 21	TCP 22
Credentials in capture	Readable in plaintext	Protected within encrypted SSH traffic
Commands / session data	FTP commands visible	Payload appears encrypted
Transfer encryption	Not provided by standard FTP	Provided by SSH
Test result	Successful transfer	Successful transfer
Integrity check	SHA-256 matched	Same sample-file workflow; hash comparison recorded

The experiment showed that both protocols could transfer the test file successfully, but they offered very different levels of protection for data in transit. The key difference was not whether the transfer worked; it was how much sensitive information could be read from the captured traffic.

11. Important Lab Notes and Credential Handling

The FTP account was created solely for this lab with a disposable password that was not reused. After the experiment, I removed the labftp account and its home directory using deluser --remove-home, then verified that the account no longer existed with id labftp.

The original FTP packet capture preserves the historical test session and therefore includes the plaintext credential observed during the experiment. Deleting the account made that credential invalid for the deleted account, but it did not erase the historical password from the capture file. The credential was unique to this lab and was not reused. This distinction is documented so that the evidence is not misrepresented.

12. Challenges Resolved

- FTP chroot restriction: corrected directory ownership and permissions so the account was confined without making its chroot root writable.

- Capture timing: started Wireshark before generating a fresh login and transfer so the capture contained the relevant packets.
- SFTP remote path: checked the remote working directory and navigated to the correct upload directory before transferring the file.
- Evidence verification: compared SHA-256 values to confirm that the transferred file matched the original.

13. Security Recommendations

- Prefer SFTP or another properly configured encrypted transfer method over standard FTP when credentials or data may cross an untrusted network.
- Use unique credentials for testing and avoid reusing personal or production passwords.
- Restrict service accounts to the directories and permissions they actually need.
- Use strong SSH configuration and keep the server software updated.
- Capture and inspect network traffic during security testing, while protecting any credentials or sensitive metadata present in the resulting evidence.
- Use hashes such as SHA-256 when an independent file-integrity check is required.

14. Limitations and Future Improvements

This was a single-VM loopback experiment. It demonstrates the protocol-level visibility difference in a controlled environment, but it does not reproduce every condition found on a real network. A useful next iteration would place the client and server on separate virtual machines and capture traffic across a virtual network interface.

The project focused on standard FTP without TLS compared with SFTP over SSH. It did not benchmark performance, test FTPS, audit SSH hardening, or assess a production deployment. Those would be separate extensions.

15. Conclusion

This project gave me practical experience configuring file-transfer services, troubleshooting service and directory permissions, capturing network traffic, interpreting protocol behaviour in Wireshark and verifying file integrity with SHA-256.

The most important result was that standard FTP exposed authentication information in readable form, whereas SFTP traffic was protected by SSH encryption. The exercise reinforced a basic security principle: a successful file transfer is not necessarily a secure file transfer. The protection of credentials and data while they travel across a network matters just as much as the transfer itself.

Appendix A. Evidence Screenshots

No.	Screenshot filename	Evidence
01	01-sample-file-creation-and-sha256-hash.png	Sample file creation and initial hash
02	02-ftp-service-running.png	FTP service running
03	03-ftp-user-creation.png	Dedicated FTP lab account creation
04	04-ftp-server-configuration-and	FTP configuration and service status

	status.png	
05	05-ftp-server-listening-port-21.png	FTP listening on TCP port 21
06	06-ftp-login-success.png	Successful FTP login
07	07-ftp-file-transfer-success.png	Successful FTP file upload
08	08-wireshark-ftp-traffic-capture.png	FTP traffic captured in Wireshark
09	09-wireshark-ftp-plaintext-credentials.png	Plaintext FTP credential exposure
10	10-ssh-server-status-running.png	SSH service running
11	11-ssh-server-listening-port-22.png	SSH listening on TCP port 22
12	12-sftp-login-success.png	Successful SFTP login
13	13-sftp-file-transfer-success.png	Successful SFTP file upload
14	14-wireshark-sftp-encrypted-traffic.png	Encrypted SSH/SFTP traffic in Wireshark
15	15-sha256-file-integrity-comparison.png	Matching SHA-256 values

The screenshots are stored in the screenshots/ directory. The packet captures are stored separately in packet-captures/. The FTP capture is retained as evidence of the plaintext exposure observed in this controlled test; its historical lab credential is no longer valid for the deleted lab account.